

Taller 1 – JavaScript: Promesas, APIs y Manipulación de Datos

Programación Web Backend / JavaScript

Objetivos

Al finalizar este taller, el estudiante será capaz de:

- Consumir información desde una API pública utilizando fetch.
- Aplicar correctamente Promise y async/await.
- Implementar consultas concurrentes utilizando Promise.all().
- Manipular colecciones de datos utilizando únicamente:
 - map
 - filter
 - find
 - some
 - every
 - reduce
- Comprender la diferencia entre ejecución secuencial y concurrente.
- Trabajar colaborativamente utilizando **GitHub Flow**.

Integrantes

Grupos de **3 estudiantes**.

Metodología de trabajo – GitHub Flow

Todo el desarrollo del taller deberá realizarse utilizando **GitHub Flow**.

Requisitos

1. Un integrante del equipo creará un repositorio en GitHub e invitará a los demás integrantes como colaboradores.
2. La rama **main** deberá permanecer siempre estable y funcional.
3. Cada integrante desarrollará una funcionalidad diferente en una rama propia.

Ejemplos:

feature/normalizacion

feature/consultas

feature/estadísticas

4. Ningún integrante podrá desarrollar directamente sobre la rama **main**.
5. Cada funcionalidad deberá integrarse mediante un **Pull Request**.
6. Antes de realizar el merge, otro integrante del equipo deberá revisar el Pull Request y dejar evidencia de la revisión (comentario o aprobación).
7. El historial del repositorio deberá evidenciar la participación de todos los integrantes mediante commits realizados desde sus respectivas ramas.

API a utilizar

API oficial de Rick and Morty:

<https://rickandmortyapi.com/api/character>

Documentación:

<https://rickandmortyapi.com/documentation>

La API entrega la información de manera paginada.

La respuesta contiene un objeto llamado **info**, el cual incluye, entre otros datos:

- total de personajes
- número total de páginas
- enlace a la página siguiente
- enlace a la página anterior

Importante

El taller deberá consultar **TODAS** las páginas disponibles de la API.

No se permite:

- consultar únicamente las primeras páginas;
- limitar la cantidad de personajes;
- copiar manualmente las URL de todas las páginas.

La solución debe identificar automáticamente el número total de páginas y recorrerlas mediante código.

Al finalizar la consulta, todos los personajes deberán almacenarse en un único arreglo.

Parte A – Normalización (map)

A partir de la información obtenida, construir un nuevo arreglo con la siguiente estructura:

```
{  
  id,  
  nombre,  
  estado,  
  especie,  
  tipo,  
  genero,  
  origen,  
  ubicacionActual,  
  cantidadEpisodios,  
  imagen  
}
```

Ejemplo

```
{  
  id: 1,  
  nombre: "Rick Sanchez",  
  estado: "Alive",  
  especie: "Human",  
  tipo: "",  
  genero: "Male",  
  origen: "Earth (C-137)",  
  ubicacionActual: "Citadel of Ricks",  
  cantidadEpisodios: 51,  
  imagen: "https://..."  
}
```

Parte B – Consultas

1. filter

Obtener todos los personajes que:

- estén vivos (Alive);
- pertenezcan a la especie **Human**.

2. filter

Obtener todos los personajes que aparezcan en **20 o más episodios**.

3. find

Encontrar el primer personaje que:

- sea de la especie **Alien**;
- tenga género **Female**.

4. some

Determinar si existe al menos un personaje cuyo campo **type** tenga información.

La respuesta deberá ser:

True o False

5. every

Verificar que todos los personajes:

- tengan imagen;
- aparezcan al menos en un episodios

6. reduce

Agrupar los personajes por especie y calcular:

- cantidad de personajes;
- promedio de episodios;
- cantidad de personajes vivos.

Ejemplo

```
{
  Human:{
    cantidad:150,
    promedioEpisodios:18.6,
    vivos:110
  },
  Alien:{
    cantidad:220,
    promedioEpisodios:7.8,
    vivos:140
  }
}
```

7. reduce

Clasificar los personajes según la cantidad de episodios en los que aparecen.

- 1–5 episodios
- 6–15 episodios
- 16–30 episodios
- Más de 30 episodios

Ejemplo

```
{
  "1-5":320,
  "6-15":180,
  "16-30":60,
  "30+":15
}
```

Parte C – Programación Asíncrona

Implementar dos soluciones para obtener la información completa de la API.

Estrategia 1 – Consultas secuenciales

Consultar las páginas utilizando await dentro de un ciclo, realizando una petición después de finalizar la anterior.

Registrar el tiempo de ejecución.

Estrategia 2 – Consultas concurrentes

Consultar todas las páginas utilizando Promise.all().

Registrar el tiempo de ejecución.

La solución definitiva del taller deberá utilizar esta estrategia.

Análisis

Responder las siguientes preguntas:

1. ¿Cuál estrategia obtuvo un mejor tiempo de ejecución?
2. ¿Qué ventajas ofrece Promise.all()?
3. ¿Qué desventajas puede tener realizar demasiadas solicitudes concurrentes?
4. ¿En qué situaciones utilizaría consultas secuenciales y en cuáles consultas concurrentes?

Entregables

Cada grupo deberá entregar:

- Código fuente completo.
- Enlace al repositorio de GitHub.
- Documento (máximo una página) respondiendo las preguntas de análisis.

Criterios de evaluación

Criterio	Porcentaje
Consumo correcto de la API (fetch, async/await)	20 %
Normalización y consultas con map, filter, find, some, every y reduce	35 %
Implementación de Promise.all() y comparación de estrategias	15 %
Aplicación correcta de GitHub Flow (ramas, Pull Requests, revisiones y commits)	20 %
Calidad del código, organización y documentación	10 %